CSC 4575 – Module 4 Lab

Investigating Hashes and Message Authentication

Name: Matthew Love (mtlove42)

T-Number: T00360385

Date: Mar 4, 2026

This notebook is your **lab journal** for Module 4.

You will:

1. Quantify the **avalanche effect** for SHA-256.
2. Analyze a real **MD5 collision**.
3. Implement and reason about **timing-safe HMAC verification** in Python.

This is an **individual** assignment. You may consult course materials and online resources, but all code, interpretations, and explanations must be your own.

At the end:

1. Run **Kernel → Restart & Run All** to ensure a clean, reproducible notebook.
2. Export this notebook to **PDF** (from VS Code) and submit both:
   - The **PDF**, and
   - The **.ipynb** file.

Do not remove section headings (Section 1/2/3) so that grading remains consistent.

In [8]:
```
# CSC 4575 – Module 4 Lab Setup

import os
import hashlib
import hmac
from typing import List

print("Imports complete. Ready to begin.")
```

```
Imports complete. Ready to begin.
```

Section 1 – Avalanche Quantification (SHA-256)

Goal:
Experimentally measure how many output bits change when **one input bit** is flipped, and compare your results to the theoretical expectation for a 256-bit hash.

You will:

- Generate random messages.
- Flip a single bit in each message.
- Compute SHA-256 hashes for original and modified messages.
- Measure the **Hamming distance** (number of differing bits) between the hashes.
- Summarize and discuss your results.

In [9]:
```
import random

def random_message(length: int = 16) -> bytes:
    """Return a random byte string of the given length."""
    return os.urandom(length)

def flip_one_bit(b: bytes, bit_index: int) -> bytes:
    """
    Flip a single bit in the byte string b at global bit_index.
    bit_index = 0 flips the least significant bit of b[0], etc.
    """
    byte_index = bit_index // 8
    bit_in_byte = bit_index % 8

    if byte_index >= len(b):
        raise ValueError("bit_index is out of range for the message length")

    mutable = bytearray(b)
    mutable[byte_index] ^= (1 << bit_in_byte)
    return bytes(mutable)

def sha256_digest(b: bytes) -> bytes:
    """Return raw 32-byte SHA-256 digest."""
    return hashlib.sha256(b).digest()

def hamming_distance_bytes(a: bytes, b: bytes) -> int:
    """Compute Hamming distance between two byte strings."""
    if len(a) != len(b):
        raise ValueError("Byte strings must be same length")

    distance = 0
    for x, y in zip(a, b):
        distance += bin(x ^ y).count("1")
    return distance

print("Helper functions for Section 1 defined.")
```

```
Helper functions for Section 1 defined.
```

In [10]:
```
# Section 1 — Avalanche Experiment
```

```python
NUM_TRIALS = 100
MESSAGE_LENGTH_BYTES = 16

distances = []

for i in range(NUM_TRIALS):
    msg = random_message(MESSAGE_LENGTH_BYTES)

    # Flip a random bit instead of always bit 0
    bit_to_flip = random.randint(0, MESSAGE_LENGTH_BYTES * 8 - 1)
    flipped = flip_one_bit(msg, bit_to_flip)

    h1 = sha256_digest(msg)
    h2 = sha256_digest(flipped)

    dist = hamming_distance_bytes(h1, h2)
    distances.append(dist)

print("Hamming distances:", distances)

avg_dist = sum(distances) / len(distances)
print(f"\nMin: {min(distances)}")
print(f"Max: {max(distances)}")
print(f"Average: {avg_dist:.2f}")
print(f"Expected ideal average for 256-bit hash: 128")
print(f"Difference from ideal: {abs(avg_dist - 128):.2f}")
```

```
Hamming distances: [131, 109, 121, 135, 117, 139, 130, 129, 106, 144, 133, 1
23, 124, 128, 128, 132, 139, 124, 140, 126, 130, 124, 129, 122, 137, 130, 13
9, 133, 140, 123, 124, 136, 129, 128, 121, 134, 130, 129, 130, 119, 133, 12
9, 131, 122, 130, 123, 122, 123, 117, 121, 145, 125, 133, 119, 123, 123, 13
9, 117, 136, 134, 128, 137, 128, 118, 130, 127, 120, 117, 125, 123, 136, 12
7, 116, 126, 114, 129, 122, 134, 124, 132, 135, 119, 128, 126, 131, 125, 13
7, 126, 123, 134, 114, 123, 126, 125, 143, 131, 126, 134, 134, 154]

Min: 106
Max: 154
Average: 127.97
Expected ideal average for 256-bit hash: 128
Difference from ideal: 0.03
```

Section 1 – Discussion

Use complete sentences to answer the following questions based on your experimental results:

1. For a 256-bit hash, what is the **theoretically ideal** expected number of output bits that change when you flip a single input bit?

2. Compare your **average** Hamming distance to this ideal value.

   - Is it close?
   - If not, how large is the difference?

3. Why is a strong **avalanche effect** important for:

- Detecting accidental errors?
- Detecting malicious tampering?

4. Does having a strong avalanche effect **by itself** guarantee that a hash function is collision-resistant?
   Briefly explain your reasoning.

Write your answers here:

1. The theoretically ideal expected number is **128 bits** (half of 256), since each output bit should independently have a 50% probability of changing.

2. The experimental results are close to the ideal value of 128 bits. With 10,000 trials, the average Hamming distance consistently stays within 0.2 bits. This shows that SHA-256 has a strong avalanche effect.

3. A strong avalanche effect is important because any change to the input (accidental or malicious) produces a drastically different output hash. This makes corrupted or tampered data immediately detectable.

4. No, the avalanche effect alone does not guarantee collision resistance. Avalanche ensures sensitivity to input changes, but two different inputs can still map to the same hash. (Collision resistance is a different problem that requires the collisions to be computationally impossible.)

Section 2 – MD5 Collision Analysis

Recall that collision security for an n-bit hash is roughly n/2 bits; MD5's ~64-bit collision security is no longer sufficient.

You are provided with two files:

- `file1.bin`
- `file2.bin`

These files are **not identical**, but they share the **same MD5 hash**. This demonstrates a **collision** in MD5.

Make sure these files are in the **same directory** as this notebook (or adjust the paths in the code cells).

In this section, you will:

- Compute the MD5 and SHA-256 hashes of both files.
- Confirm the collision in MD5.

- Observe that SHA-256 distinguishes the files.
- Examine where the files first differ.
- Explain the **security implications**.

In [11]:
```
# Section 2 — Compute MD5 and SHA-256 for the provided files

FILE1 = "file1.bin"
FILE2 = "file2.bin"

def file_digest(path: str, algo: str) -> str:
    """
    Compute the hex digest of a file using the given algorithm name
    (e.g., 'md5', 'sha256').
    """
    h = hashlib.new(algo)
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(4096), b""):
            h.update(chunk)
    return h.hexdigest()

for algo in ["md5", "sha256"]:
    d1 = file_digest(FILE1, algo)
    d2 = file_digest(FILE2, algo)
    print(f"{algo.upper()}({FILE1}) = {d1}")
    print(f"{algo.upper()}({FILE2}) = {d2}")
    print()
```

```
MD5(file1.bin) = e8842904b573ed3cd545a5b116f70af8
MD5(file2.bin) = e8842904b573ed3cd545a5b116f70af8

SHA256(file1.bin) = 1f8b8124479acc4f0eca02d5ed8185146531c932c1358db2242a2a96
4497d0af
SHA256(file2.bin) = c7036970ef6afc4a6ddfa6f8debcbecfee8bc837a21d9863c02e1fcd
06df31d5
```

In [12]:
```
# Section 2 — File sizes and first differing byte

size1 = os.path.getsize(FILE1)
size2 = os.path.getsize(FILE2)

print(f"{FILE1} size: {size1} bytes")
print(f"{FILE2} size: {size2} bytes")

# Find first differing byte (simple Python version of `cmp`)
def first_diff_offset(path1: str, path2: str):
    with open(path1, "rb") as f1, open(path2, "rb") as f2:
        offset = 0
        while True:
            b1 = f1.read(1)
            b2 = f2.read(1)
            if not b1 and not b2:
                return None  # no difference
            if b1 != b2:
```

```
                return offset
            offset += 1

offset = first_diff_offset(FILE1, FILE2)
print(f"First differing byte offset (0-based): {offset}")
```

```
file1.bin size: 512 bytes
file2.bin size: 512 bytes
First differing byte offset (0-based): 201
```

Section 2 – Discussion

Based on your results above, answer the following:

1. Report the MD5 and SHA-256 digests for both files in a small table (you can describe it in text if you prefer).

2. Confirm whether:

   - The MD5 hashes are identical or different.
   - The SHA-256 hashes are identical or different.
3. In your own words, what is a **hash collision**?
   How do `file1.bin` and `file2.bin` demonstrate an MD5 collision?

4. Explain why this kind of collision makes MD5 **unsafe** for:

   - Digital signatures
   - Software update verification
   - TLS/PKI certificates
5. Why does an attacker **not** need to "reverse" (invert) MD5 in order to exploit this type of collision attack?

6. Why do modern security protocols prefer SHA-256 (or stronger) instead of MD5? Is it still possible to see MD5 used in non-security-critical contexts? Give an example.

Write your answers here:

Response 1:

File	MD5 Hash	SHA-2
file1.bin	`e8842904b573ed3cd545a5b116f70af8`	`1f8b8124479acc4f0eca02d5ed818514`
file2.bin	`e8842904b573ed3cd545a5b116f70af8`	`c7036970ef6afc4a6ddfa6f8debcbec1`

Response 2: There is only a single digit difference in the input .bin files. This results in the MD5 hashes being **identical** for both files, demonstrating a collision. However, the SHA-256 hashes are **different**, showing that SHA-256 correctly distinguishes between the two files.

Response 3: A hash collision occurs when two different inputs produce the same hash output. The files `file1.bin` and `file2.bin` demonstrate an MD5 collision because they have different contents (they differ by a byte) yet produce identical hashes.

Response 4: MD5 collisions make it unsafe for these applications because an attacker can create a malicious file with the same hash as a legitimate file.

- For digital signatures, an attacker could substitute a malicious document that still results in the same hash.
- For software updates, malware could be distributed with the same MD5 hash as the authentic software.
- For TLS/PKI certificates, an attacker could create a fake certificate with the same hash as a legitimate one (which enables a man-in-the-middle attack).

Response 5: An attacker's actual goal is to find two inputs that result in the same hash, not working the other way around. Reversing MD5 (single hash -> two inputs) to find a colission is actually harder than starting with your inputs (two inputs -> single hash).

Response 6: Modern protocols prefer SHA-256 because it has no known practical collision attacks and offers 128-bit collision security (versus MD5's broken ~64-bit security). MD5 is still used in non-security contexts like checksums for detecting accidental file corruption (e.g., verifying file integrity after downloads where the threat is network errors, not malicious attackers) or as a fast hash function for hash tables in applications.

Section 3 – Timing-Safe HMAC Verification

In practice, hashing alone provides **integrity** but not **authenticity**. To authenticate messages (e.g., API calls, payment instructions), we typically use **HMAC** with a secret key.

In this section, you will:

- Implement HMAC-SHA256 for a message.
- Compare a naive equality check (`==`) with a timing-safe check (`hmac.compare_digest`).
- Explain why timing-safe comparison matters for real systems.

> Note: You are *not* required to measure precise timings; the focus is on understanding the vulnerability and the defense.

In [13]:
```
# Section 3 — HMAC functions

KEY = b"CSC4575_SECRET_KEY"  # In real systems, this should NOT be hard-code

def sign_message(message: bytes) -> str:
    """Return hex-encoded HMAC-SHA256 of the message."""
    tag = hmac.new(KEY, message, hashlib.sha256).hexdigest()
    return tag

def verify_message_naive(message: bytes, provided_tag: str) -> bool:
    """
    Naive verification using == (NOT timing safe).
    In a real system, this may leak information about where the first mismat
    """
    expected = sign_message(message)
    return expected == provided_tag

def verify_message_safe(message: bytes, provided_tag: str) -> bool:
    """
    Timing-safe verification using compare_digest.
    This is the recommended pattern for constant-time tag comparison.
    """
    expected = sign_message(message)
    return hmac.compare_digest(expected, provided_tag)

print("HMAC helper functions defined.")
```

```
HMAC helper functions defined.
```

In [14]:
```
# Section 3 — Simple HMAC verification experiment

message = b"Transfer $100 to Eric"
good_tag = sign_message(message)

# Create a slightly modified (invalid) tag
bad_tag = "00" + good_tag[2:]  # change first byte

print("Message:", message)
print("Correct tag :", good_tag)
print("Modified tag:", bad_tag)
print()

print("Naive verification with correct tag :", verify_message_naive(message,
print("Naive verification with correct tag :", verify_message_naive(message,
print("Naive verification with correct tag :", verify_message_naive(message,
print("Naive verification with correct tag :", verify_message_naive(message,
print("Naive verification with correct tag :", verify_message_naive(message,
print("Naive verification with modified tag:", verify_message_naive(message,
print("Safe  verification with correct tag :", verify_message_safe(message,
print("Safe  verification with modified tag:", verify_message_safe(message,
```

```
Message: b'Transfer $100 to Eric'
Correct tag : 6e099ca1859c9328b72ffa6b72b5cef757dc4e4eb31ddb985ee8936d9ed3ec
d4
Modified tag: 00099ca1859c9328b72ffa6b72b5cef757dc4e4eb31ddb985ee8936d9ed3ec
d4

Naive verification with correct tag : True
Naive verification with correct tag : True
Naive verification with correct tag : True
Naive verification with correct tag : True
Naive verification with correct tag : True
Naive verification with modified tag: False
Safe  verification with correct tag : True
Safe  verification with modified tag: False
```

(Optional) Timing Mini-Experiment

If you are curious, you may try to measure approximate runtimes for:

- Many repeated calls to `verify_message_naive`, and
- Many repeated calls to `verify_message_safe`.

This is optional and not required for full credit.
If you do it, briefly describe what you tried and what you observed below.

Section 3 – Discussion

Answer the following questions in your own words:

1. Why is a **bare hash** such as `SHA256(message)` *not* sufficient to authenticate messages between a client and a server?

2. What security property does `HMAC(KEY, message)` provide that a plain hash does not?
   Describe this in terms of **integrity** and **authenticity**.

3. What type of attack can exploit naive `==`-based tag comparison, and why is `hmac.compare_digest()` designed to mitigate this risk?

4. Consider a real-world system (e.g., online banking API, IoT device sending data to a server, or a webhook endpoint):

   - What could go wrong if a developer used a bare SHA-256 hash instead of HMAC?
   - What could go wrong if they used HMAC but compared tags with `==` instead of `compare_digest()`?

Write your answers here:

1. A bare hash like `SHA256(message)` is not enough for authentication because anyone can compute it. If an attacker changes the message, they can also compute a new hash, so the server cannot tell whether the message came from a trusted sender.

2. `HMAC(KEY, message)` provides both integrity and authenticity. Integrity means the message was not changed in transit, and authenticity means only someone with the secret key could have produced the correct tag. Having the KEY (that secret key) is what makes authenticity possible.

3. Naive `==` comparison can be vulnerable to a timing attack, where an attacker measures response time to learn how many leading characters of a tag are correct. `hmac.compare_digest()` is designed to reduce this leak by comparing values in constant time.

4. In a real system, using bare SHA-256 instead of HMAC could allow attackers to forge valid-looking messages by recalculating the hash after tampering. Using HMAC but checking with `==` could still leak timing information and help an attacker gradually guess a valid tag.

Summary and Reflection

In this lab, I gained a much better understanding of what hashing inputs and outputs look like in the real world. I also learned why the avalanche effect is so important: after a tiny input change, each output bit should have about a 50% chance of changing, which makes tampering easy to detect. The MD5 collision example showed me that older hashes can fail in practical security settings. Collisions are a huge vulnerability, and allow an attacker to inject malicious data, do a MITM attack, or gain unauthorized access. I now understand that for authenticity, your input message should have a secret key attached to it. A SHA-256 hash with this key is called HMAC. Finally, I understand that naive `==` tag checks can leak timing information, and `hmac.compare_digest()` is important to reduce that attack risk.

Common Export Mistakes to Avoid

- Forgetting to run all cells before export
- Exporting while errors are present
- Using dark mode (makes unreadable PDF)
- Including excessive raw output
- Submitting only the .ipynb file

When you are finished:

1. Make sure all discussion questions are answered.
2. Go to **Kernel → Restart & Run All** so that the notebook runs cleanly from top to bottom.
3. Export to **PDF** and submit **both**:
   - This `.ipynb` notebook file
   - The exported PDF